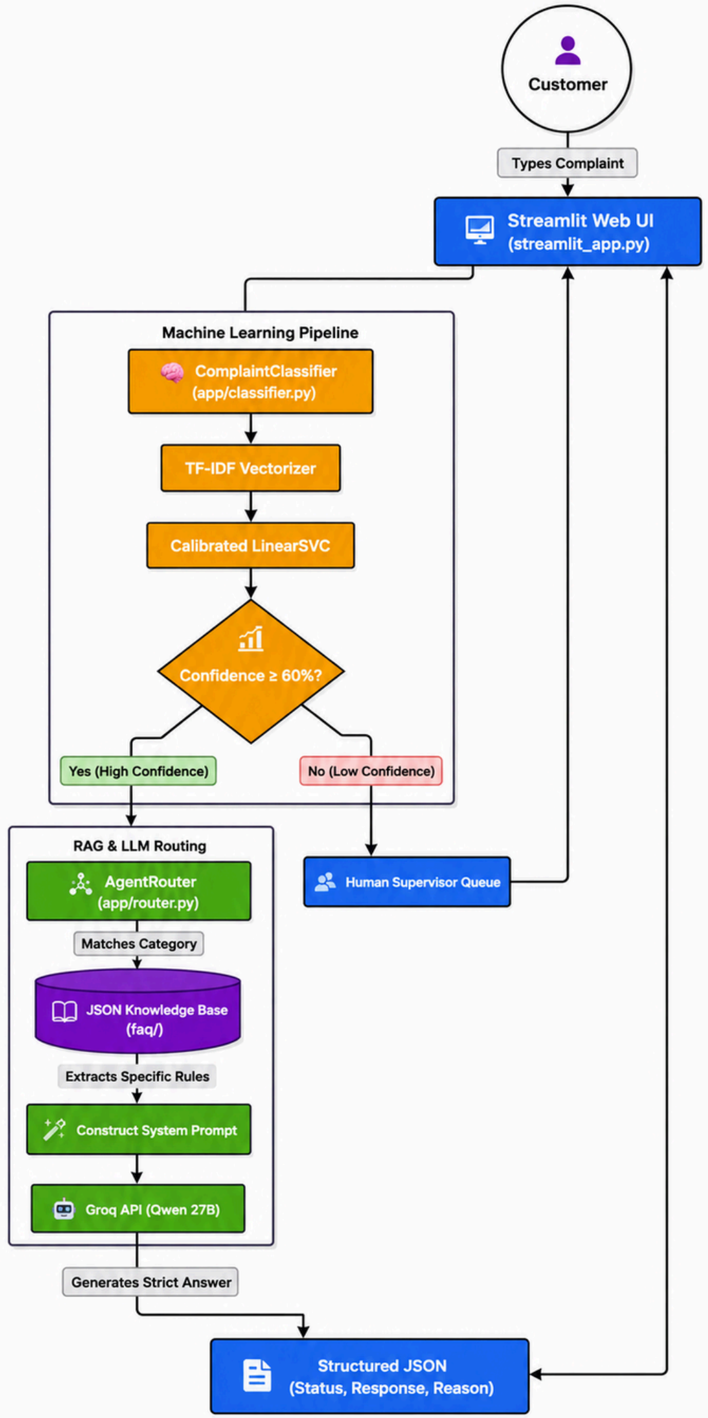
NATURAL LANGUAGE PROCESSING PIPELINE

Customer Complaints Classification

An NLP system that classifies customer complaints into product categories and routes uncertain cases for manual review

PROJECT TEAM

Mostafa Mohamed • Mohamed Emam • Mohamed Ali • Mohamed Sherif • Mohamed Medhat



System Architecture

The system follows a hybrid AI architecture for automated customer complaint handling. The complaint is submitted through the Streamlit Web UI and classified using a TF-IDF + Calibrated LinearSVC pipeline. Based on the model's confidence, high-confidence complaints are processed through RAG, where relevant rules are retrieved from a JSON knowledge base and provided to the Qwen 27B LLM to generate a structured response. Low-confidence cases are instead escalated to a human supervisor to ensure reliability and prevent incorrect automated decisions.

INTRODUCTION

System Overview

System Overview

Customer Complaints Classification is an automated text-classification system designed to read raw consumer complaint narratives and predict the relevant financial product.

Automated prediction enables real-time routing to target specialized teams without manual sorting overhead.

Target Categories

`credit_card` `retail_banking` `credit_reporting`

`debt_collection` `mortgages_and_loans`

SMART FALLBACK MECHANISM

When the model is not confident enough in a prediction, the complaint is routed to "**Other Problem**" for manual review instead of being force-classified.

DATA CONTEXT

Project Overview & Dataset

Project Scope

A multi-class NLP classification pipeline trained on real consumer complaint narratives, built to automatically route each complaint to its correct product category with confidence thresholding.

Dataset Metadata

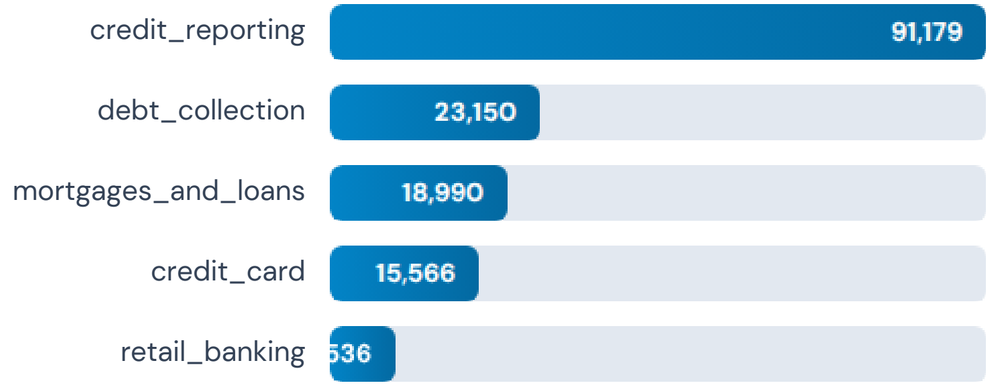
Source: CFPB Complaints (complaints_processed.csv)

Columns: product, narrative

Raw Volume: 162,421 records

Classes: 5 primary product categories

Category Distribution (Raw Data)



ARCHITECTURE

Workflow Phases

1 **Environment Preparations**
Setup Jupyter environment & libraries

2 **Reading & Cleaning Data**
Missing value removal & deduplication

3 **Feature Engineering**
Text normalization & TF-IDF vectorization

4 **Exploratory Data Analysis**
Class distributions & length analysis

5 **Applying AI Models**
Classical ML vs Deep Learning models

6 **Deployment**
Inference pipeline with threshold routing

PHASE 1

1. Environment Preparations

Technical Setup

All technical execution and experiments were conducted within interactive **Jupyter Notebooks (.ipynb)**.

Core numerical computation, data manipulation, serialization, and string parsing modules were initialized upfront.

CORE DEPENDENCIES:

`pandas` `numpy` `joblib` `re (regex)`

</> CLASSIFIER.PY IMPORTS

```
import re
import joblib
import numpy as np

# Environment initialized successfully
```

PHASE 2

2. Reading & Cleaning Data

Data Preprocessing Steps (01_data_exploration.ipynb)

- Dropped redundant index column (Unnamed: 0)
- Removed records missing narrative text (10 null rows)
- Eliminated exact duplicate complaints (37,735 duplicate rows)
- Filtered out short placeholder text (narratives ≤ 10 characters, e.g., "name")

162,421
RAW ROWS LOADED

162,411
POST DROPNA

124,620
FINAL CLEANED DATASET SIZE

PHASE 3

3. Feature Engineering

Text Cleaning

Function: clean_text

- Convert text to lowercase
- Remove redacted tokens ("xxxx")
- Remove non-alphabetic characters
- Collapse extra spaces

Train / Test Split

Stratified Partitioning

80% Training Set

20% Test Set

random_state = 42

Stratified by target product

TF-IDF Vectorizer

Feature Parameters

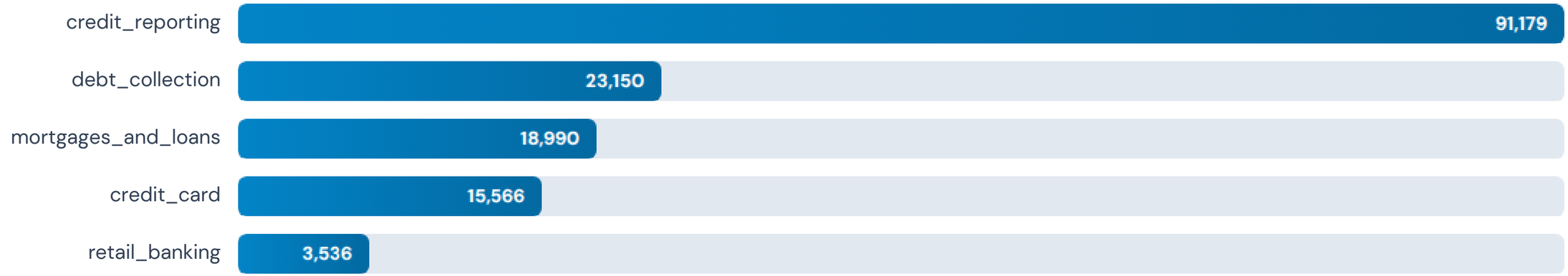
- max_features: 10,000
- ngram_range: (1, 2)
- min_df: 5 | max_df: 0.8
- sublinear_tf: True

PHASE 4

4. Exploratory Data Analysis

Category Distribution

Significant class imbalance was identified during EDA, with credit reporting dominating the consumer complaints.



PHASE 5

5. Applying AI Models

Target: Complaint product category. Multiple supervised ML models were trained on TF-IDF features alongside a Deep Learning model on raw sequences.

Classical ML Models

- Logistic Regression
- Logistic Regression (Balanced)
- LinearSVC (Balanced)
- Multinomial Naive Bayes
- **Calibrated LinearSVC** – via CalibratedClassifierCV

Deep Learning Architecture

model_2.ipynb

- Embedding Layer (128 dims)
- Bidirectional LSTM (64 units)
- Bidirectional LSTM (32 units)
- Dense (64, ReLU) + Dropout (0.5)
- Dense (Softmax) Output Layer

PHASE 5 — DEEP LEARNING DEEP DIVE

DL Architecture Rationale (model_2.ipynb)

🧠Why RNN / LSTM?

Complaint narratives are inherently **sequential text**. Unlike TF-IDF which treats narratives as order-agnostic bags of words, Recurrent Neural Networks (specifically LSTMs) preserve word order and maintain long-term contextual syntax across full paragraphs.

↔Bidirectional Layer

A Bidirectional LSTM processes the narrative simultaneously **forward and backward**. Each token's final state captures context from words that came BEFORE and AFTER it—critical because key phrases indicating the true category can appear anywhere in long complaints.

🔗Stacked BiLSTM (64 → 32)

The first **BiLSTM(64)** layer extracts broad contextual syntax across sequence steps. The second **BiLSTM(32)** layer compresses these sequences into a higher-level abstract feature representation before feeding into the Dense classification head.

EVALUATION

Model Training & Evaluation

DL Training Setup

Tokenizer: max_words = 10,000 | max_len = 200
Optimizer: Adam | Loss: sparse_categorical_crossentropy
Batch size: 32 | Max Epochs: 20 (Early Stopping patience=5)

Evaluation Approach

Evaluated using Classification Report, Confusion Matrix, and Accuracy / Precision / Recall / F1-Score per model across all 5 financial product classes.

Model	Accuracy	F1-Score
Logistic Regression (balanced)	84%	0.84
LinearSVC (Balanced)	86%	0.86
Multinomial NB	82%	0.82
Calibrated LinearSVC (Production Selected) ⭐	86%	0.86
BiLSTM (DL)	85%	0.85

MODEL EVALUATION COMPARISON

Classical ML vs Deep Learning (BiLSTM)

Evaluation Dimension	Calibrated LinearSVC (Best Classical ML)	Stacked BiLSTM (Deep Learning)
Accuracy / F1-Score	**86% / 0.86** (Slightly superior generalization)	**83% / 0.83** (Highly competitive)
Training & Compute Cost	Lightweight & Ultra-fast: Trains on CPU in seconds; low memory footprint during inference.	Heavy Compute: Requires GPU acceleration, long training epochs, and larger runtime memory.
Context Handling	TF-IDF n-grams (1-2 words). Relies on word frequency, ignoring global word order.	Captures full sequential word dependencies and contextual context both forward and backward.
Interpretability	High: Feature weights directly reveal top predictive keywords per category.	Low ("Black Box"): Complex non-linear neural activations hard to interpret directly.

Final Selection Decision: Calibrated LinearSVC was chosen for production deployment due to its superior accuracy, minimal CPU latency, lower infrastructure overhead, and capability to provide calibrated probability scores needed for routing.

PHASE 6

6. Deployment

Inference Pipeline (classifier.py)

1. Load calibrated_svc_model.pkl & tfidf_vectorizer.pkl
2. clean_text(): lowercase, strip redactions/symbols
3. vectorizer.transform() → TF-IDF vector
4. model.predict_proba() → class probabilities
5. **Routing Decision:** If max probability ≥ 0.70 → predicted class, otherwise → **"Other Problem"**

```
# EXAMPLE INFERENCE CALL

classifier = ComplaintClassifier(
  model_path="models/calibrated_svc_model.pkl",
  vec_path="models/tfidf_vectorizer.pkl"
)

decision, confidence = classifier.classify(
"I have been charged a late fee on my visa but I paid it on
time!"
)
```

PHASE 6 — TECHNICAL IMPLEMENTATION

Deployment Implementation Details

Role of CalibratedClassifierCV

Standard LinearSVC outputs uncalibrated decision function scores (distance to hyperplane) rather than true probabilities. Wrapping LinearSVC inside **CalibratedClassifierCV (Platt Scaling)** transforms raw scores into well-calibrated posterior probabilities, making the **0.70 confidence threshold** mathematically meaningful.

Production Considerations

Models are serialized with joblib and initialized **once at application startup** (cached in memory) to prevent disk I/O latency per request. The pipeline is designed to be easily encapsulated into a lightweight REST API (FastAPI) or a Streamlit UI for real-time classification.

Step-by-Step Inference Flow

1. Raw Text
Input complaint narrative received

2. Clean Text
Lowercase, remove redactions & symbols

3. TF-IDF Vector
Transform text to feature matrix

4. Predict Proba
Calculate probability distribution

5. Threshold Check
Prob ≥ 0.70 → Category
Prob < 0.70 → Other Problem

SUMMARY

Conclusion

1

End-to-End NLP Pipeline

Built an end-to-end NLP pipeline that classifies customer complaints into 5 product categories from the CFPB dataset.

2

Comprehensive Model Comparison

Compared multiple classical ML models (Logistic Regression, LinearSVC, Naive Bayes) alongside a Bidirectional LSTM deep learning model.

3

Reliable Production Routing

Deployed a calibrated model with a confidence threshold, ensuring uncertain complaints are safely routed for manual review rather than misclassified.

Thanks

Customer Complaints Classification Presentation

Questions & Discussion